INGESTION · DOMAIN 2 - DATA INGESTION AND ACQUISITION

Auto Loader in depth

Section 2. Schema evolution, rescued data, file detection, and one pipeline for batch and streaming.

SECTION 2 of 10 WEIGHT 7% TOPIC Auto Loader

admin@sthithapragna.com

Auto Loader is a stream that remembers which files it has read. **Everything else follows from that memory.**

How exactly-once actually works

- File metadata is persisted in a **key-value store inside the checkpoint**, not by listing what already landed.
- So a file is processed once, and a restart resumes rather than reprocessing.
- It scales to **billions of files** for a backfill, and **millions of files per hour** ongoing.
- Delete the checkpoint and that memory is gone. Every file looks new again.

> ON THE EXAM
>
> This is why Auto Loader beats a hand-written 'list the directory and compare' loop, which gets slower as the directory grows.

What you actually configure

OPTION	PURPOSE
`cloudFiles.format`	The real source format. Always set it.
`cloudFiles.schemaLocation`	Where the inferred schema is stored and updated.
`cloudFiles.schemaEvolutionMode`	What happens on a new column.
`cloudFiles.schemaHints`	Override the inferred type for named columns.
`cloudFiles.rescuedDataColumn`	Name of the column holding what did not parse.
`cloudFiles.useManagedFileEvents`	Use the managed file events service.

ON THE EXAM

Schema location and checkpoint location are separate settings storing different things. A production stream needs both.

Five modes, and what each does to the stream

MODE	BEHAVIOUR ON A NEW COLUMN
`addNewColumns`	Stream fails. Column added to the schema. Existing types unchanged.
`rescue`	Schema never evolves, stream never fails. New columns go to rescued data.
`failOnNewColumns`	Stream fails. No restart until you update the schema or remove the file.
`none`	New columns ignored, nothing rescued unless asked. Stream does not fail.
`addNewColumnsWithTypeWidening`	As addNewColumns, plus safe type widening. Unsafe changes rescued.

ON THE EXAM

Notice that addNewColumns fails the stream. The failure is part of the evolution, not an alternative to it.

Why addNewColumns failing is not a bug

- The stream stops with an unknown field error when a new column appears.
- **Before it throws**, it infers from the latest micro-batch and **updates the schema location**, merging new columns to the end.
- On **restart**, the evolved schema is in place and the stream carries on.
- So Databricks recommends running Auto Loader as a job configured to **restart automatically**.
- Without automatic restart, the stream sits failed and somebody has to notice.

> ON THE EXAM
>
> This is the module 9 point exactly: schema evolution assumes retries are configured. The retry is the second half of the mechanism.

The default depends on whether you supplied a schema

- **No schema provided:** the default is **addNewColumns**.
- **Schema provided:** the default is **none**.
- addNewColumns is **not allowed** alongside a provided schema at all.
- It does work if you supply your schema as a **schema hint** instead.

> ON THE EXAM
>
> Widely-repeated blog posts claim the default is to fail, like failOnNewColumns. The documentation says otherwise. Trust the docs.

What lands in the rescued column

- Three causes: a **column missing from the schema**, a **type mismatch**, or a **case mismatch**.
- It holds a **JSON blob** containing the rescued columns and **the source file path** of the record.
- That file path is what makes it possible to trace a bad record back to its origin.
- Case differences are rescued by default. Set the reader to case-insensitive to change that.

> ON THE EXAM
>
> Nothing is lost and nothing errors, which is the point. It is also why a column full of rescued data can go unnoticed for months.

Telling it what you already know

- Override the inferred type for named columns, without supplying a whole schema.
- The right tool when you know a column should be a timestamp and inference keeps calling it a string.
- For Parquet they do **not cast**. They tell the reader to read the column as that type, and a mismatch is rescued.
- They also let you keep addNewColumns while still pinning the columns you care about.

> ON THE EXAM
>
> Hints are the middle ground between full inference and a hard-coded schema. They fix what you know without freezing what you do not.

Directory listing against file events

DIRECTORY LISTING

The default

- Lists the input directory to find new files
- No extra permissions beyond reading the data
- Optimised, and detects lexical ordering
- Cost grows with the size of the directory

FILE EVENTS

The recommendation

- One managed service per external location
- One shared queue serving many consumers
- Backfill and tuning handled for you
- Enable on the location, then set the option

REASSURING

You can switch detection modes across stream restarts and keep exactly-once guarantees. It is not a one-way decision.

Options it makes unnecessary

- **Backfill interval** is handled automatically for locations with file events enabled.
- **Fetch parallelism** is tuned for you. There is no manual knob.
- The choice between incremental listing and notifications disappears. There is one mode.
- Notification cleanup is handled, including when a stream is deleted or fully refreshed.

> ON THE EXAM
>
> If an answer proposes tuning fetch parallelism or setting a backfill interval on a file-events stream, it is describing an older configuration.

Run it at least once a week

- The file events service **caches recently created files**. That cache expires.
- Databricks recommends running Auto Loader streams **at least once every seven days**.
- Otherwise the stored read position becomes invalid and a **full directory listing** is required to resync.
- A full listing also happens when starting a new stream, or migrating from another detection mode.

> ON THE EXAM
>
> Correctness is not at risk here. The cost is a slow first run after a long gap, which matters when the directory is enormous.

One definition, batch or streaming

- The **same Auto Loader definition** runs continuously, or as a batch with the available now trigger.
- The **checkpoint carries over**, so a batch run picks up exactly where the last one stopped.
- That is how one pipeline serves an hourly schedule today and continuous ingestion later, with no rewrite.
- Writing to Delta makes the target **append-only and transactional**, which is what downstream streams expect.

> ON THE EXAM
>
> This is the section 2 objective: an append-only pipeline handling both batch and streaming data. It is one definition and a trigger choice.

What to carry into the exam

- **addNewColumns fails the stream**, updates the schema first, and resumes on restart. Configure automatic retries.
- Default is **addNewColumns** without a schema, **none** with one. Hints avoid that trap.
- Rescued data catches **missing columns, type mismatches and case mismatches**, with the source file path.
- **File events** is the recommendation. Run streams at least weekly or pay for a full listing.
- Same definition, batch or streaming. The trigger decides, and the checkpoint carries over.

> ON THE EXAM
>
> That completes Section 2 at 7%. Section 3 is transformation and data quality, at 10%.

> An Auto Loader stream ingests JSON events with no schema supplied. A producer adds a new field. The stream fails overnight with an unknown field error and an engineer restarts it manually the next morning, after which it runs normally and the new column is present.
>
> This has happened four times this quarter. The team believes Auto Loader is unreliable.

What should they conclude and do?

A The failure is part of how addNewColumns evolves the schema. Configure the job to restart automatically.

B Switch to failOnNewColumns so the schema is locked and cannot change unexpectedly.

C The schema location is corrupt. Delete it and let inference start again.

D Supply an explicit schema so no inference happens and the stream never fails.

◆ PAUSE AND DECIDE

What should they conclude and do?

A The failure is part of how addNewColumns evolves the schema. Configure the job to restart automatically.

The schema is updated before the error is thrown, so the restart is the second half of the mechanism.

B Switch to failOnNewColumns so the schema is locked and cannot change unexpectedly.

Makes it strictly worse. That mode will not restart at all without manual intervention.

C The schema location is corrupt. Delete it and let inference start again.

It is working correctly. The restart proves the updated schema was written.

D Supply an explicit schema so no inference happens and the stream never fails.

Switches the default to none, so new fields are silently ignored rather than ingested.

ON THE EXAM

The clue is that a plain restart fixes it every time and needs nothing else. That is a handover, not a fault.

A finance team notices a quarterly report is missing a revenue adjustment field that the source system has been sending for two months. The Auto Loader stream has never failed and reports healthy throughput.

The stream was configured with an explicit schema when it was built, to guarantee stable column types.

What is happening?

A The field is in the rescued data column, because a type mismatch prevented parsing.

B Providing a schema set the evolution mode default to none, so new columns are ignored.

C The schema location has not been updated because the checkpoint was cleared.

D The stream is using failOnNewColumns and has been silently skipping the offending files.

◆ PAUSE AND DECIDE

What is happening?

A The field is in the rescued data column, because a type mismatch prevented parsing.

Plausible for a type problem. Here the column is entirely new, and none mode ignores it without rescuing.

B Providing a schema set the evolution mode default to none, so new columns are ignored.

The two-defaults rule. With a schema supplied, new columns are ignored and nothing is rescued unless asked.

C The schema location has not been updated because the checkpoint was cleared.

A cleared checkpoint reprocesses files. It does not silently drop a column.

D The stream is using failOnNewColumns and has been silently skipping the offending files.

That mode fails loudly. Nothing about it is silent.

ON THE EXAM

The team made a reasonable decision for a good reason and inherited a default they never chose. That is the shape of this trap.

A regulated pipeline must never ingest a record it has not validated, and any unexpected field must stop the pipeline for review before data reaches the target. Uptime is secondary to correctness.

The team currently uses rescue mode because it was described as the safest option.

Which mode fits the requirement?

A. Keep rescue mode; unexpected fields are captured in rescued data and can be reviewed later.

B. Use failOnNewColumns, which stops the stream until the schema is deliberately updated.

C. Use addNewColumns, which fails the stream on each new column.

D. Use none, so unexpected fields are ignored and never reach the target.

◆ PAUSE AND DECIDE

Which mode fits the requirement?

A Keep rescue mode; unexpected fields are captured in rescued data and can be reviewed later.

Data keeps flowing to the target, which is exactly what the requirement forbids.

B Use failOnNewColumns, which stops the stream until the schema is deliberately updated.

It will not restart until a human updates the schema or removes the file. That is the review gate the requirement asks for.

C Use addNewColumns, which fails the stream on each new column.

It fails, then evolves and resumes on restart. The new field is ingested without review.

D Use none, so unexpected fields are ignored and never reach the target.

Nothing stops, and nobody reviews anything.

ON THE EXAM

Safest depends on what you are protecting. Rescue protects uptime; failOnNewColumns protects the target.

A stream ingests CSV files with an explicit rescued data column enabled. A downstream analyst reports that an entire customer identifier column is now null, starting from a specific date. The rescued data column has values for those rows.

The upstream team confirms they made a cosmetic change: the header now reads CustomerID rather than customerid.

What happened, and what is the least disruptive fix?

A. A case mismatch routed the field to rescued data. Set the reader to case-insensitive.

B. The column was renamed, so a full refresh is needed to rebuild the schema.

C. A type mismatch occurred because the new header changed the inferred type.

D. The schema location must be deleted so inference picks up the new header.

◆ PAUSE AND DECIDE

What happened, and what is the least disruptive fix?

A A case mismatch routed the field to rescued data. Set the reader to case-insensitive.

Case mismatch is one of the three documented rescue causes, and the case-insensitive reader option addresses it directly.

B The column was renamed, so a full refresh is needed to rebuild the schema.

Nothing was renamed in substance, and a full refresh is far more disruptive than a reader option.

C A type mismatch occurred because the new header changed the inferred type.

A header's capitalisation does not change a column's type.

D The schema location must be deleted so inference picks up the new header.

Would work, discards the accumulated schema, and leaves the same trap for next time.

ON THE EXAM

Case is the third rescue cause and the one people forget. A purely cosmetic upstream change can empty a column.

> A quarterly compliance extract runs an Auto Loader stream against a bucket holding several million files. It is triggered four times a year. Each run now takes hours before it begins processing, though it processes very little data.
>
> The stream uses file notification mode with file events enabled on the external location.

What explains the slow start?

A. The file events cache has expired, so Auto Loader performs a full directory listing to resynchronise.

B. File events are not supported for quarterly schedules and the stream has fallen back permanently.

C. The checkpoint has grown too large to load efficiently after three months.

D. Backfill interval must be set manually when a stream runs less than daily.

◆ PAUSE AND DECIDE

What explains the slow start?

A The file events cache has expired, so Auto Loader performs a full directory listing to resynchronise.

Streams should run at least once every seven days. A quarterly run is far outside that, so the read position is invalid.

B File events are not supported for quarterly schedules and the stream has fallen back permanently.

There is no such restriction, and the fallback is per-run rather than permanent.

C The checkpoint has grown too large to load efficiently after three months.

Checkpoint size tracks files processed, and this stream processes very little.

D Backfill interval must be set manually when a stream runs less than daily.

Backfill is handled automatically for locations with file events enabled.

ON THE EXAM

Correctness is unaffected here. The symptom is cost and time, which is what makes it hard to diagnose from the output alone.

A team ingests partner files into a bronze Delta table. Today the business wants an hourly schedule to keep costs down. Within a year they expect to move to continuous ingestion for a near-real-time dashboard.

They want to avoid rewriting the ingestion when that happens, and no data may be reprocessed at the switchover.

What should they build now?

A. A batch job reading the directory and comparing against the target to find new files.

B. An Auto Loader stream with the available now trigger, scheduled hourly, switching the trigger later.

C. COPY INTO on an hourly schedule, replaced by an Auto Loader stream when continuous is needed.

D. A continuous Auto Loader stream now, throttled with a long trigger interval to control cost.

◆ PAUSE AND DECIDE

What should they build now?

A. A batch job reading the directory and comparing against the target to find new files.
Hand-built bookkeeping that gets slower as the directory grows, and a full rewrite later.

B. An Auto Loader stream with the available now trigger, scheduled hourly, switching the trigger later.
One definition, one checkpoint. Changing the trigger is not a rewrite, and the checkpoint prevents reprocessing.

C. COPY INTO on an hourly schedule, replaced by an Auto Loader stream when continuous is needed.
Works today and forces the rewrite they asked to avoid, with no checkpoint to carry over.

D. A continuous Auto Loader stream now, throttled with a long trigger interval to control cost.
Keeps compute running continuously, which is the cost they were trying to avoid.

ON THE EXAM

This is the Section 2 objective in a scenario: one append-only pipeline serving batch today and streaming later.